CM SMR (D2IFC000100)
Pablo Martínez Pavón
Curso 2026/2027

IES de Teis

# B2.5 — Gimnasio: listeners

## Módulo DI – UD2

*Nome e apelidos do alumn@*

---

En la B2.4 reaccionaste a **eventos**: algo que el usuario hace en un momento concreto (un clic, una tecla). En esta tarea reaccionarás a **cambios de valor**. Casi todo lo que muestra un control es una **propiedad** observable: el texto de un campo (`textProperty()`), si una casilla está marcada (`selectedProperty()`), qué opción de un grupo está elegida (`selectedToggleProperty()`)… A una propiedad se le puede añadir un **listener**, un código que se ejecuta **cada vez que cambia su valor**, lo cambie el usuario o el propio programa.

La aplicación es el formulario de alta del **gimnasio**, con estética de película de espías de los años 60. La interfaz y los estilos ya están hechos: **solo escribirás código Java** en los puntos marcados con `// TODO`.

## 1. Proyecto de partida

Descarga y abre la carpeta `di-b2-5` en VS Code y configura JavaFX.

```
di-b2-5/
└── src/
    └── app/
        ├── Main.java      ← aquí trabajarás
        └── mi6.css        ← terminada
```

Ejecuta la aplicación. El formulario se ve, pero no reacciona: el contador no cambia, el total no se actualiza y **Registrar** está siempre habilitado.

**Métodos terminados que vas a usar:**

| Método | Qué hace |
|---|---|
| `calcularTotal()` | Lee la cuota elegida y la casilla de piscina, y escribe el total en `lblTotal`. |
| `correoValido(texto)` | Devuelve `true` si el texto tiene forma de correo (`algo@algo.algo`). |
| `mostrarAyuda(etiqueta, texto, clase)` | Cambia el texto de una etiqueta de ayuda y le pone la clase `correcto`, `incorrecto` o ninguna (`null`). |
| `limpiar()` | Deja el formulario como al principio, **desde código**. |

# Cómo se escribe un listener

```
txtCiudad.textProperty().addListener((obs, viejo, nuevo) -> {
    System.out.println("Antes: " + viejo + " · Ahora: " + nuevo);
});
```

La lambda recibe tres parámetros: la propiedad observada (`obs`, casi nunca se usa), el valor **anterior** y el valor **nuevo**. Los nombres los eliges tú.

## 2. Contador de caracteres

**Qué debe pasar:**

- Mientras se escribe el nombre en clave, `lblContador` muestra cuántos caracteres lleva: `3/30`.
- No se pueden escribir más de 30 caracteres. Si se supera el límite, el campo recupera el texto anterior.

**Ejemplo.** Este listener, de otra aplicación, pasa a mayúsculas el código postal mientras se escribe:

```
txtCodigo.textProperty().addListener((obs, viejo, nuevo) -> {
    txtCodigo.setText(nuevo.toUpperCase());
});
```

**Pista:** el límite está en la constante `MAX_NOMBRE`. Para recuperar el texto anterior, usa el parámetro `viejo`.

**Pregunta:** al hacer `setText(viejo)` dentro del listener, el texto vuelve a cambiar. ¿Se vuelve a ejecutar el listener? ¿Por qué no se produce un bucle infinito?

## 3. Correo comprobado mientras se escribe

**Qué debe pasar:**

| Texto del campo | Etiqueta | Borde del campo |
|---|---|---|
| Vacío | Sin texto | Normal |
| Correo válido | `Correo válido`, clase `correcto` (verde) | Normal |
| Correo no válido | `Formato no válido`, clase `incorrecto` (rojo) | Rojo |

El borde rojo lo pone la hoja con la **pseudo-clase propia** `:error`. Ya está definida en `Main` como la constante `ERROR`. Una pseudo-clase propia se activa y desactiva desde Java:

```
txtTelefono.pseudoClassStateChanged(ERROR, true);    // activa :error
txtTelefono.pseudoClassStateChanged(ERROR, false);   // la desactiva
```

**Pregunta:** ¿por qué con el campo vacío no se muestra ningún error, aunque un texto vacío no sea un correo válido?

## 4. Total mensual

**Qué debe pasar:** el total se recalcula en cuanto se elige otra cuota o se marca o desmarca la piscina.

Las tres cuotas son `RadioButton` del mismo `ToggleGroup` (`grupoCuota`). El grupo tiene la propiedad `selectedToggleProperty()`, que cambia cuando se elige otra opción. Basta con un listener en el grupo, no uno en cada botón.

**Ejemplo.** Este listener muestra la talla elegida en un grupo de tallas:

```
grupoTalla.selectedToggleProperty().addListener((obs, anterior, actual) -> {
    lblTalla.setText("Talla: " + ((RadioButton) actual).getText());
});
```

**Pista:** el cálculo ya está hecho en `calcularTotal()`. Solo tienes que llamarlo desde dos listeners.

## 5. Registrar solo con los datos completos

**Qué debe pasar: Registrar** solo está habilitado si el nombre no está vacío, el correo es válido y se ha marcado la casilla de las condiciones.

1. Completa el método `actualizarBoton()`. Usa `setDisable(...)`.
2. Llama a `actualizarBoton()` cada vez que cambie cualquiera de los tres datos: desde los listeners de T1 y T2, y desde un listener nuevo en `chkCondiciones`.

**Pregunta:** pulsa **Limpiar** con el formulario relleno. ¿Se actualizan el contador, el total y el botón? ¿Por qué, si nadie ha escrito ni pulsado nada en esos campos?

**Pregunta:** para la casilla de condiciones también podrías haber usado `setOnAction`. Cuando `limpiar()` ejecuta `chkCondiciones.setSelected(false)`, ¿se ejecutaría ese manejador? ¿Y el listener? Compruébalo con un `System.out.println` en cada uno.